Fog-Tier Windowing and a Serverless Cloud Backend for Live Electric-Vehicle Charging-Hub Monitoring

Nemi Ishwarlal Vikani
Student ID: X24303046
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x24303046@student.ncirl.ie

***Abstract*—Electric-vehicle charging hubs draw heavy, fluctuating current, hold battery packs within a narrow thermal band, and load a shared local grid connection, yet the state of a charging bay is often inferred from occasional manual checks rather than continuous measurement, so an overcurrent condition, an overheating cabinet, or a grid-load spike can escalate unseen between inspections. This paper presents a continuous two-hub charging-network monitoring pipeline. Five metric types per hub feed a fog gateway that windows and aggregates readings and evaluates safety thresholds close to the equipment, forwarding only compact summaries onward. A serverless cloud backend built on Amazon SQS, AWS Lambda, and Amazon DynamoDB ingests those summaries, and a second function serves a live operations dashboard through Amazon API Gateway, with the static frontend hosted on Amazon S3. Each layer is justified against the alternatives that were weighed and set aside, and the system is deployed to a real cloud account rather than only a local emulator. An automated suite of 121 tests passes, and verification against the live deployment confirmed every pipeline health check reporting healthy, fresh readings arriving within seconds, the stored record count climbing, and both hubs' metrics together with two firing safety alerts rendering correctly on the deployed dashboard.**

***Keywords*—electric-vehicle charging, fog computing, edge computing, Internet of Things, serverless computing, Amazon SQS, AWS Lambda, Amazon DynamoDB**

I. Introduction

An electric-vehicle charging hub is an increasingly dense electrical worksite. Fast chargers push tens of amperes into vehicle battery packs, those packs must be kept inside a narrow temperature band to charge safely and preserve their life [1], and every bay in a hub draws on one shared grid connection whose aggregate load has to stay within the site's contracted capacity. As charging demand grows, coordinating that demand across a distribution network without overloading it has become a distinct engineering problem, and the smart-charging literature treats the visibility of each site's real-time electrical state as a precondition for managing it [2]. When that visibility comes from periodic manual inspection rather than continuous measurement, an overcurrent fault at a bay, a cabinet drifting toward an over-temperature condition, or the hub's combined draw climbing past its safe grid limit can each develop in the interval between one check and the next.

Instrumenting a charging hub with always-on sensors closes that interval, but it raises the same distributed-systems question every dense sensing deployment faces: where the readings are processed and how they travel from the equipment to the people who act on them. Streaming every raw sample from every bay to a distant region wastes bandwidth on values that individually rarely matter and delays exactly the moment a threshold breach needs to be raised. Fog computing was proposed to address this by placing an intermediate compute tier between the sensors and the cloud, close enough to the equipment to summarise, filter, and evaluate rules before anything crosses the wide-area network [3], and it is one expression of the broader movement of computation back toward the network edge for latency-sensitive and bandwidth-heavy workloads [4]. Behind that fog tier, the cloud supplies the on-demand, elastically provisioned model that lets a backend absorb bursts and sit idle at no cost without anyone pre-sizing a server [5].

This project builds that two-tier design for a scaled-down but structurally faithful charging network: two hubs, designated hub-1 and hub-2, each carrying the same five metric types (charging current, battery state of charge, station temperature, grid load, and session duration) generated by ten independent sensor containers. A fog gateway ingests their readings, windows and aggregates each hub-and-metric group on a fixed interval, evaluates threshold rules against each aggregate, and forwards only the finished window and any alerts onward. A cloud-side pipeline persists that window, and a dashboard folds the persisted history into a per-hub operational view alongside a cross-hub grid-load trend.

The work sets four objectives, each to be demonstrated running rather than only described. First, a sensor and fog layer that genuinely windows and alerts across five metric types at independently configurable sampling and dispatch rates instead of a rate fixed in code. Second, a backend that scales through managed, elastic cloud primitives rather than a single always-on server. Third, a dashboard that turns the

stored windows into an operational reading a hub operator could act on without inspecting raw values. Fourth, and distinct from the local emulator most development iterates against, deployment onto a real public-cloud account and verification directly against the live endpoint. The scope is deliberately bounded to these layers at a two-hub scale rather than a full multi-site charging network. The remainder of this paper is organised as follows. Section II sets out the architecture and the reasoning behind each component choice, including the alternatives weighed and the security posture. Section III covers the implementation, the automated test suite, continuous integration, the real deployment together with the pre-deployment audit, and the evidence gathered from the running system. Section IV closes with a critical assessment of the design's trade-offs, its limitations, and where the work could go next.

II. System Architecture and Design

Fig. 1 shows the deployed system as three tiers: an edge tier where ten sensor containers and the fog gateway share one host, a cloud ingest pipeline of a queue, an ingestion function, and a store, and a cloud serving tier where a second function answers a browser through an API gateway and a static bucket.

A. Sensor and Fog Layer

Each of the ten sensor containers represents one metric type on one hub. A container advances a bounded random walk from its previous value on each sample, clamped to a physically plausible range for its metric, so consecutive readings stay close together like a real instrument trace rather than independent noise; this is what makes the windowed minimum, maximum, and average, and the dashboard's trend line, resemble genuine charging telemetry. Every container is driven by two independent, separately configurable intervals, one governing how often a new value is sampled and the other how often the accumulated batch is dispatched to the fog gateway, so a fast-changing quantity such as charging current can be sampled frequently while still being sent in economical batches. Within a container the two duties are structured as two recurring one-shot tasks that re-arm themselves through a completion callback on a small thread pool, so the thread pool itself owns the recurrence rather than a central scheduling loop, and the shared reading buffer is swapped out as a whole when a dispatch begins, so readings sampled during the network call accumulate in a fresh buffer instead of racing the request in flight. If a dispatch fails, the drained batch is simply prepended back onto the buffer for the next attempt, so a transient network fault delays readings rather than dropping them.

The fog gateway is a small Python web application with no schema-validation framework in its request path; posted batches are parsed by hand and checked by a separate, directly testable validation routine that is deliberately permissive about the metric name, so a new charging-bay metric can be added without editing it. Accepted readings are buffered per hub-and-metric group under a single lock. On each fixed window boundary the gateway snapshots and clears the whole buffer under that lock, then, outside the lock so a slow publish never blocks an incoming reading, reduces each non-empty group to a compact summary carrying the count, minimum, maximum, average, and latest value, and evaluates that summary against the network's threshold rules. Four rules are held as plain data, each naming a metric, the aggregate field to read, a comparison, a limit, and the alert to raise: a station-temperature average above its limit raises an overheating alert, a charging-current average above its limit raises an overcurrent alert, a grid-load average above its limit raises a grid-strain alert, and a session-duration average above its limit raises a stalled-session alert. The fifth metric, battery state of charge, deliberately carries no rule; it is one of the five required signals and is shown on the dashboard as a secondary reading, but it never alerts, since a state-of-charge value is informational rather than a fault condition at hub level. A descriptive catalogue of the rules is exposed on a read-only endpoint, rebuilt from the same rule data on every call so it can never drift from what is actually enforced.

B. Backend: Queue, Function, and Store

The fog gateway never calls the ingestion function directly. Every window's summaries are placed on an Amazon SQS queue instead, the decoupling that lets a producer and a consumer whose rates should vary independently proceed without either blocking the other, a pattern distributed messaging systems were built around [6]. A direct synchronous call would tie the gateway's throughput to whatever function concurrency happened to be free at that instant, and a burst that exceeded it would fail with nowhere for the data to wait; the queue absorbs that burst as backlog instead. Publishing is batched: an entire window's summaries are collected and sent together, split only where the queue interface caps a batch at ten entries, so the number of publish calls tracks the number of groups that closed in a window rather than how many individual readings arrived, which holds the per-message overhead down as volume grows.

An event-driven function consumes the queue: it is invoked with a batch of messages, transforms each summary into a stored record, and writes it to Amazon DynamoDB. The store is keyed so that the partition key is the metric type and the sort key joins the window-end time with the hub identifier. Leading the sort key with the window-end time makes the store's own key ordering chronological, so the dashboard's dominant query, the most recent windows for a metric type, is a single backward read within a partition rather than a whole-table scan; appending the hub identifier is deliberate, because the two hubs closing a window for the same metric in the same cycle would otherwise produce an identical key and the second write would silently overwrite the first. DynamoDB was chosen for exactly the access pattern its key-value lineage was designed around, predictable key-based reads and writes at scale without index tuning as the table grows, the lineage decentralised wide-column stores established for high-write, partition-oriented workloads [7]. Both functions run on

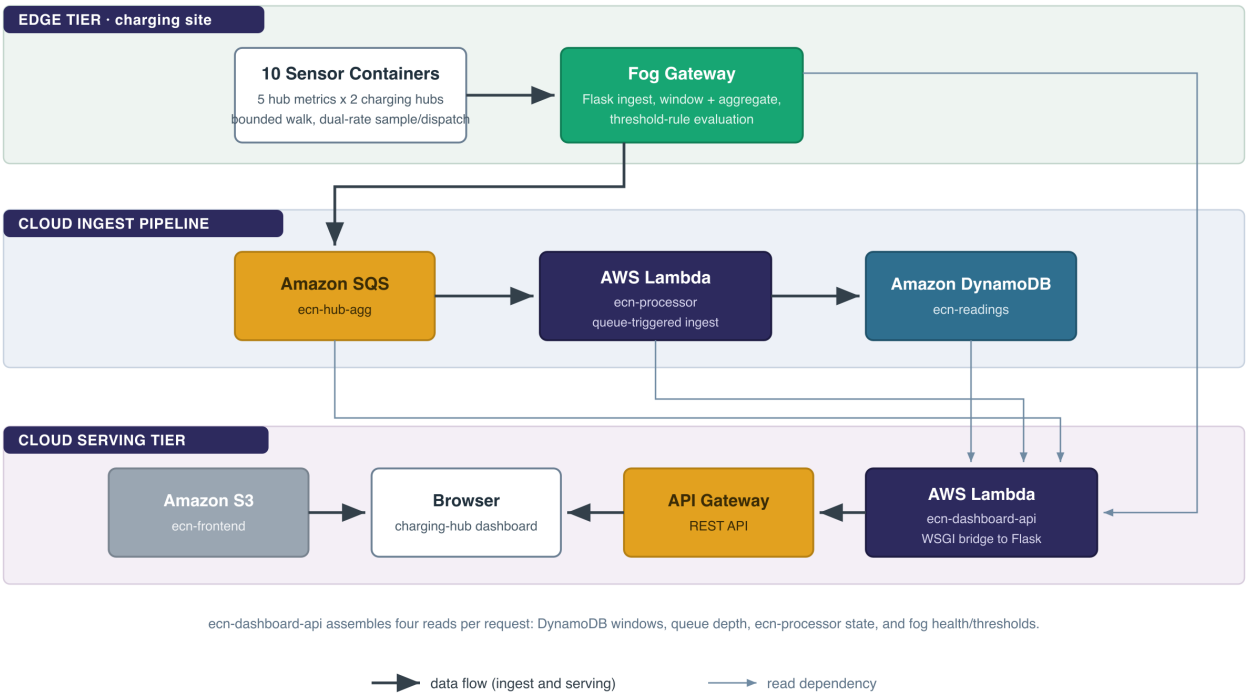

Fig. 1. Deployment topology drawn as three tiers. In the edge tier the ten sensor containers dispatch to the fog gateway, which windows and evaluates rules. The ingest pipeline runs left to right: the gateway publishes to the queue, the ingestion function drains it into the store. In the serving tier the browser is fed from both sides, static assets from the S3 bucket and live data from the dashboard function through the API gateway. The thin lines are the four read dependencies the dashboard function assembles per request: the stored windows, the queue depth, the ingestion function's deployment state, and the fog gateway's health and threshold endpoints.

demand rather than continuously: the ingestion function fires once per delivered batch and the dashboard function once per request, and neither is billed while idle, which suits a workload that only produces a message when a metric group actually reported during a window. The recognised cost of that elasticity is cold-start latency on an idle function [8], but the ingestion path is asynchronous and queue-triggered rather than user-facing, so a cold start delays only when a reading is durably stored, never a page load, and the dashboard's own polling keeps its function warm.

C. Dashboard API and Frontend

The operator view is served by a second function, deployed independently of the ingestion function so the read path and the write path scale and fail separately. It answers a small read-only interface: a per-hub grouping endpoint that returns, for each metric type, the latest window reported by each hub; a readings endpoint that returns a recent series for one metric to draw the cross-hub trend; a health endpoint; a backend-statistics endpoint; and the descriptive rule catalogue proxied from the fog gateway and cached after its first fetch, since the rules are static for the deployment's life. Rather than re-declaring those routes for the cloud runtime or pulling in a third-party adapter, the function is a small bridge that adapts an incoming cloud-function request into the standard Python web-server interface [9] and invokes the existing application object unchanged, so every route the local server already serves answers identically behind the API gateway from one implementation. Every response the function returns, including its not-found and error paths, carries an unconditional wildcard cross-origin header, because the frontend is served from a storage-bucket origin while the interface answers from a separate gateway origin and a browser would otherwise block the cross-origin reply without surfacing a visible network error.

The health endpoint reasons about pipeline freshness by reading four independent things directly rather than trusting one cached flag: whether the fog gateway answers, whether the queue is reachable, whether the ingestion function reports itself deployed, and how old the freshest stored window is across every metric type. That last check is the one that actually detects a stalled pipeline, because a queue and a function can both look healthy while no new data has landed for an hour, and only a timestamp comparison against the store catches it. The backend-statistics endpoint reports the queue's waiting and in-flight depths and the stored record count; that count is computed by reading a count-only scan across every page the

store returns rather than a single call, so it stays correct once the scanned data grows beyond the size a single scan response is capped at.

The static frontend renders each hub as a card of five metric rows drawn against native meter elements, a set of alert tags naming every active breach on that hub, a cross-hub grid-load trend chart drawn with a locally vendored charting library rather than one loaded from a content-delivery network, and a four-indicator pipeline health strip. A fixed browser-side timer re-fetches the per-hub snapshot, health, backend statistics, and trend series together on a short interval and repaints from whatever those responses hold, so the view stays current without a manual reload or any push channel. The frontend learns which interface origin to call from a single inert element in the page whose link target is substituted with the real gateway origin at upload time and read once by the page script, a mechanism chosen so the same static files can be uploaded against any deployment's endpoint without a rebuild.

D. Critical Analysis of Alternatives

Several designs were weighed and set aside, each for a stated reason rather than by default. A managed streaming log was considered in place of a plain queue: it would keep an ordered, replayable record of every window, which a standard queue does not guarantee, but ordering and replay are unnecessary here because the dashboard reads the latest window per metric from the store directly, not a stream of intermediate states [6], so a streaming log would have added partition-management overhead with no matching need to spend it on. A relational database was considered in place of the key-value store and set aside for this workload: the access pattern is a narrow, known set of key-based reads and appends, not the ad-hoc joins a relational engine is built for, and choosing the managed key-value store also avoids provisioning, patching, or paying for an always-on instance, keeping the backend serverless end to end. For the dashboard runtime, re-declaring the interface routes a second time for the cloud function, or adding a third-party framework adapter, was considered against reusing the existing application through the standard web-server interface; the reuse was chosen because a parallel set of routes would be a standing source of drift between the local and cloud behaviours, and the standard interface makes that reuse a few lines rather than a dependency [9]. Finally, converting the sensor and fog tier itself to a scheduled-function model, so the whole system and not only the backend would be serverless, was considered and explicitly deferred rather than dismissed: a continuous sensor loop and a stateful windowed buffer map awkwardly onto a stateless, time-limited invocation model without reworking the buffering, a mismatch that reflects a known limitation of first-generation serverless platforms for stateful, data-centric work [8], and since the deployment scope here is the backend layer, this is recorded as future work rather than attempted partially.

E. Security Posture

The edge host exposes only a single inbound port carrying the fog gateway's health and threshold endpoints; there is no remote-login key pair and no inbound path for interactive access, and administration goes through the cloud provider's systems-management channel, which removes the most common remote-access surface rather than merely narrowing it. What that single port serves is read-only, non-sensitive status, never data access or a control-plane operation. Both functions run under the laboratory account's single shared role because the account's policies block creating new, more narrowly scoped roles: this is a constraint of the environment, not a design preference, and a production account would grant the ingestion function only the queue-receive and item-write permissions it needs and the dashboard function only read-oriented permissions. The dashboard interface is intentionally unauthenticated because it exposes only aggregated, non-personal charging telemetry, well below the threshold where access control would be meaningful at this scale. Every cloud-facing component relies on the platform's default credential resolution, so on the real host and inside both functions the credentials come from the instance profile or the function's execution role, and no static credential pair is ever built for the production path.

III. Implementation

A. Software Stack

The entire pipeline is written in Python 3.12 across four independently packaged components: the sensors, the fog gateway, the ingestion processor, and the dashboard. The two web-facing components use a minimal web framework for routing and the standard library alone for outbound calls, with no schema-validation framework anywhere in the request path. The cloud software development kit supplies the queue, store, and function clients. The dashboard component additionally produces a second, cloud-function entry point alongside its local development server, so the same repository, pipeline-check, and threshold-proxy logic serves both a laptop and a real gateway integration without a parallel implementation.

B. Automated Test Suite

All 121 tests across the four components pass, run with a standard Python test runner against hand-written fakes of the cloud client interfaces, with no mocking library and no test touching a real cloud endpoint or a running emulator. Table I breaks the suite down by component. Coverage is weighted toward the components with the most decision logic: the fog component carries the bulk of it, exercising the buffer under interleaved ingest and flush, the window arithmetic, the threshold rules at and around their limits, the batched publisher's chunking behaviour, and the validation routine's rejection of malformed batches. Several of the fog tests bind the real web application to an ephemeral port and drive it with actual requests over a socket rather than calling a handler in process, which is the only way to prove that a body that is not valid data at all, as opposed to a well-formed body

TABLE I
AUTOMATED TEST SUITE BY COMPONENT (121 TESTS TOTAL)

Component	Tests	Coverage focus
sensors	21	bounded random walk, dual-rate sample and dispatch, atomic buffer swap, payload shape
fog	61	window arithmetic, threshold rules at boundaries, batch validation, batched publishing, HTTP ingest over a real socket
backend/processor	10	message-to-item transform, sort-key construction, batch handling
backend/dashboard	29	per-hub grouping, paginated item count, cross-origin header, request bridging, health and backend-statistics

missing a field, is rejected by whichever layer parses it first. The dashboard tests exercise the routes through the cloud-function entry point with request-shaped events and injected fake clients, asserting on the returned status, headers, and body directly, and they confirm both that the wildcard cross-origin header is present on every response and that a genuine route is reached and its result translated back into the cloud-function response shape.

C. Continuous Integration

A hosted continuous-integration workflow runs on every push and pull request as two dependent jobs rather than one. The first installs a Python 3.12 toolchain and runs the whole test suite, so a failure in any component's own tests fails the build before the second job starts. The second job runs only once the first has passed: it brings up the full stack with a container orchestrator against a local cloud emulator and runs an end-to-end verification script against the running containers, confirming that a reading actually reaches the store through the whole pipeline rather than only that individual functions return correct values in isolation, then tears the stack down regardless of outcome so no containers linger between runs.

D. AWS Deployment and Pre-Deployment Audit

Every resource the system runs on was provisioned by a single reusable infrastructure-as-code apply, twenty-four resources created in one operation with no manual command-line step in between: the store, the queue, both functions and the event-source mapping that connects the queue to the ingestion function, the full gateway chain, the edge host with its security group and fixed public address, and the two storage buckets with the frontend bucket's public-access configuration and object upload [10]. Table II lists the live resources. Declaring the whole cloud tier as one versioned artifact, rather than a sequence of console or command-line actions, means the resource topology is reproducible and the dashboard function's fog-endpoint configuration is wired automatically to the address the apply itself allocated, with no value copied by hand between steps.

A code audit carried out before deployment, rather than after, examined the codebase for the failure modes that a local

TABLE II
LIVE AWS RESOURCES (US-EAST-1)

Resource	Identifier	Role
DynamoDB table	ecn-readings	stores closed-window summaries
SQS queue	ecn-hub-agg	decouples fog gateway from the processor
Lambda	ecn-processor	queue-triggered ingestion into the store
Lambda	ecn-dashboard-api	dashboard interface behind the gateway
API Gateway REST API	8fu3l2spz0	public entry point for the dashboard interface
EC2 instance	i-04cbd11ce27a03023	fog gateway + 10 sensor containers
Elastic IP	52.202.199.193	fixed address for the edge host
S3 buckets	ecn-frontend-350600740537 / ecn-deploy-350600740537	static dashboard / deploy staging

emulator's small fixtures do not surface, and confirmed each was already handled correctly. The stored-record count is read across every scan page rather than from a single response, so it does not undercount once the table's scanned data grows past the size one scan is capped at. Publishing already batches a whole window's summaries into as few calls as the ten-entry limit allows rather than sending one message per group. And every cloud-facing component already relies on the platform's default credential resolution rather than constructing a static credential pair, so no fabricated credential can shadow the real instance-profile or execution-role identity on the deployed account. Because these were verified in the audit rather than discovered in production, no correction was needed before the stack went live.

E. Live Deployment Verification

Verification took place against the live endpoint, not only against the local suite. Within about a minute of the edge host completing its container bootstrap, the health endpoint reported all four fields true, with the freshest window a few seconds old, meaning a reading generated on a container had already traversed the sensor, the fog gateway, the queue, the ingestion function, and the store, and was being read back. The backend-statistics endpoint showed the stored record count climbing across successive polls, from a few dozen items to several hundred within minutes, rather than a static number, and the per-hub endpoint returned real per-metric data for both hubs. All four static frontend assets returned successfully from the storage bucket, the wildcard cross-origin header was present on the interface responses, and the frontend's inert origin element had been correctly substituted with the real gateway origin at upload time rather than left as its placeholder, confirming the configuration mechanism took effect on the deployed files. Fig. 2 shows the deployed dashboard rendering both hubs' telemetry with two safety alerts firing on hub-1, an overheating alert against a station-temperature

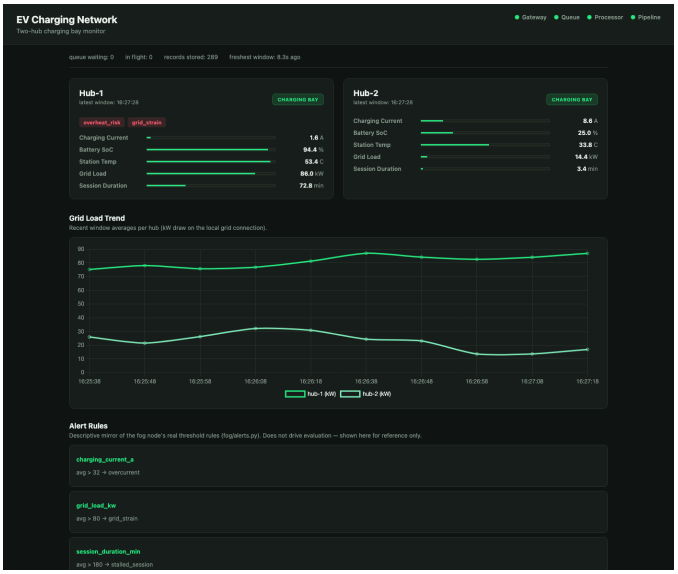

Fig. 2. The deployed dashboard rendering live two-hub telemetry through the gateway: the four-indicator health strip all healthy, both hubs' five metric rows, two firing safety alerts on hub-1 (overheating and grid strain), and the cross-hub grid-load trend. The static page and the interface it calls are both served from the serverless tier, independent of the edge host.

average past its limit and a grid-strain alert against a grid-load average past its, which together exercise the full path from a threshold evaluated at the fog tier to a badge rendered in the browser.

The implementation, spanning the sensor, fog, processor, and dashboard components and the infrastructure-as-code that provisions the live account, is available at [GitHub repository URL].

IV. Conclusion

This project set out to move charging-hub awareness from occasional inspection toward continuous, automated observation, and to do so with a distributed design that places computation where it belongs: windowing and rule evaluation at a fog tier beside the equipment, durable storage and elastic serving in the cloud. All four objectives were met. The sensor and fog layer generates five metric types per hub at independently configurable rates and reduces them to windowed summaries with real threshold evaluation rather than forwarding raw samples. The backend scales through queue decoupling, event-driven functions billed only per invocation, and an on-demand store, and each of those choices was weighed against a named alternative rather than adopted by default. The dashboard turns the stored windows into a per-hub operational reading with two safety alerts firing. And the whole pipeline was deployed to a real public-cloud account and verified there directly, not left as a description of one.

Two points stand out from the build. The first is that a passing test suite and a correct deployment are different claims: the failure modes the pre-deployment audit checked for, a paginated count that stops at the first page, per-message publish overhead, and a fabricated credential shadowing the real one, are exactly the kind that a small local fixture never exercises, so they have to be reasoned about against production scale rather than left to a test to catch. The second is architectural: keeping the dashboard and its interface fully serverless and independent of the edge host means the operator view keeps serving stored telemetry even when the edge host is paused between sessions, a property a design that rested the dashboard on the same host would not have.

The design also carries real limitations worth stating rather than glossing over. The sensor and fog tier runs on a single edge host, which is a single point of failure for that tier even though everything downstream of it is redundant managed infrastructure; the deferred move to a scheduled-function fog model would close that gap. The laboratory account is session-based, so the evidence here reflects a single continuous session rather than sustained multi-day operation, and the concurrency ceiling of the live queue and functions under a sustained burst has not yet been characterised. Both functions share one broadly scoped role because the account cannot mint narrower ones, so the least-privilege split described in Section II is specified but not deployed. Beyond closing those gaps, natural extensions are a compound rule that reasons over more than one metric at once, so that a hub simultaneously borderline on charging current and on grid load is escalated rather than shown as two independent low-severity states, and per-metric window durations so a fast-moving quantity such as charging current can be checked more often than a slow one such as session duration without forcing every metric onto the same cadence.

References

[1] L. Lu, X. Han, J. Li, J. Hua, and M. Ouyang, "A review on the key issues for lithium-ion battery management in electric vehicles," *Journal of Power Sources*, vol. 226, pp. 272–288, 2013.

[2] J. García-Villalobos, I. Zamora, J. I. San Martín, F. J. Asensio, and V. Aperribay, "Plug-in electric vehicles in electric distribution networks: A review of smart charging approaches," *Renewable and Sustainable Energy Reviews*, vol. 38, pp. 717–731, 2014.

[3] S. Yi, C. Li, and Q. Li, "A survey of fog computing: Concepts, applications and issues," in *Proc. 2015 Workshop on Mobile Big Data (Mobidata '15)*, Hangzhou, China, 2015, pp. 37–42.

[4] G. Premsankar, M. Di Francesco, and T. Taleb, "Edge computing for the Internet of Things: A case study," *IEEE Internet of Things Journal*, vol. 5, no. 2, pp. 1275–1284, 2018.

[5] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility," *Future Generation Computer Systems*, vol. 25, no. 6, pp. 599–616, 2009.

[6] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB '11)*, Athens, Greece, 2011.

[7] A. Lakshman and P. Malik, "Cassandra: A decentralized structured storage system," *ACM SIGOPS Operating Systems Review*, vol. 44, no. 2, pp. 35–40, 2010.

[8] J. M. Hellerstein, J. M. Faleiro, J. E. Gonzalez, J. Schleier-Smith, V. Sreekanti, A. Tumanov, and C. Wu, "Serverless computing: One step forward, two steps back," in *Proc. 9th Biennial Conf. Innovative Data Systems Research (CIDR '19)*, Asilomar, CA, USA, 2019.

[9] P. J. Eby, "PEP 3333 – Python Web Server Gateway Interface v1.0.1," Python Software Foundation, Tech. Rep., 2010.

[10] Y. Brikman, *Terraform: Up and Running: Writing Infrastructure as Code*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.